3

Artificially Intelligent Machine

3.1 Defining Intelligence

Howard Gardner, the American developmental psychologist, has categorized intelligence into the following categories.

Linguistic intelligence is the ability to speak, recognize and use mechanisms of phonology (speech sounds), syntax (grammar) and semantics (meaning). It is mainly exhibited by narrators and orators.

Musical intelligence is the ability to create, communicate with and understand different sounds, pitch and rhythm. This intelligence is used by musicians, singers and composers.

Logical-mathematical intelligence helps a person to use and understand complex and abstract ideas. Mathematicians and scientists extensively use this type of intelligence.

Spatial intelligence is the ability to perceive visual or spatial information, change it and re-create visual images. Using this, 3D images can be created and manipulated (translated, transformed and rotated). This intelligence is generally applied by map readers, astronauts and physicists.

Bodily-kinesthetic intelligence defines the ability to use the entire or part of the body to solve problems or manipulate objects. For example, this kind of intelligence is mostly used by players and dancers.

Intrapersonal intelligence is the ability to distinguish between one's feelings, intentions and motivations. A few people like Gautam Buddha has this instinct.

Interpersonal intelligence is the ability to recognize and differentiate between other people's feelings, beliefs and intentions. Mass communicators and interviewers use these skills to the maximum.

A machine or a system is said to be **artificially intelligent** when it exhibits at least one and at most all intelligences.

3.2 Components of Intelligence

Intelligence is an intangible entity composed of the following (**Fig. 3.1**):

- Reasoning
- Learning
- Problem solving
- Perception
- Linguistic intelligence

FIGURE 3.1 Components of Intelligence

Reasoning

It is the set of processes that is used in making decisions and prediction. There are two types of reasoning—inductive and deductive. The differences between these two techniques is given in **Table 3.1**.

TABLE 3.1 Inductive vs Deductive Reasoning

Learning

It is a process of gaining knowledge or skill by studying, practicing, being taught or experiencing something. Learning is an ability that is possessed by humans, animals and AI-enabled systems to improve the awareness of the subjects of the study.

Learning can be categorized as follows.

1. **Auditory learning** promotes learning by listening and hearing. For example, students hear recorded lectures to understand a particular concept.
2. **Episodic learning** promotes learning by remembering the sequences of events that one has witnessed or experienced. This is linear and orderly.
3. **Motor learning** helps to learn by precise movement of muscles. For example, picking objects.
4. **Observational learning** triggers learning by watching and imitating others. For example, children often learn by mimicking their parents.
5. **Perceptual learning** is done by recognizing stimuli that one has seen before. For example, identifying and classifying objects and situations.
6. **Relational learning** involves learning to differentiate among various stimuli on the basis of relational properties, rather than absolute properties. For example, if we know that the last time we cooked a dish, it had excess spices, the next time, we would add spices in lesser quantity.
7. **Spatial learning** is done through visual stimuli such as images, colours, maps, etc. For example, we usually create a roadmap in our

minds before actually moving to the road.

8. **Stimulus-response learning** facilitates subjects to perform a particular behaviour when a certain stimulus is received. For example, we usually shout when we touch a hot vessel.

Problem Solving

It is the process in which one tries to find the desired solution in the present situation by following the path which is blocked by known or unknown hurdles. Problem solving uses **decision-making to** select the best suitable alternative to reach the desired goal.

Perception

It is the process of acquiring, interpreting, selecting and organizing sensory information. While humans use sensory organs to perceive their environment, AI systems use data acquired by the sensors to do the same.

Linguistic Intelligence

It is used in in interpersonal communication and defines one's ability to use, comprehend, speak and write the verbal and written language.

3.3 Differences Between Human and Machine Intelligence

1. Humans perceive by patterns, whereas machines perceive by analysing data with respect to a given set of rules.
2. Humans store and recall information by patterns but machines use searching algorithms to do the same.
3. Humans have intelligence to deduce any information that is missing or distorted but machines lack this ability to do it with a high level of accuracy.

3.4 Agent and Environment

Artificial intelligence is all about studying rational agents, which may include anything that makes decisions. For example, a person, firm, machine or software that performs action(s) with the best outcome after considering past and current percepts at a given instance are examples of

agents. These agents work in their environment and may interact with other agents in that environment. Thus, we can say that an AI system comprises an **agent and its environment**.

FIGURE 3.2 AI Agent

In an artificially intelligent system, the agents act in their environment which may contain other agents. This **agent** (refer **Fig. 3.2**) can be anything that is capable of perceiving its environment. For this, it may have **sensors. Agents also have effectors** that help them to act upon their environment**. *The different types of agents in an AI system include the following:***

1. H**uman agent:** A human agent has sensory organs (like eyes, ears, nose, tongue and skin) that acts as sensors, and other organs such as hands, legs, mouth as effectors for taking actions.
2. R**obotic agent:** A robotic agent uses cameras and infrared range finders as sensors, and various motors and actuators as effectors.
3. S**oftware agent:** Such an agent uses bit strings as its programs and actions.

3.4.1 Key Terminology

Performance Measure of Agent

This criteria helps to determine how successful an agent is.

Behaviour of Agent

It is an action performed by an agent after any receiving a certain percept.

Percept

These are perceptual inputs given to an agent at a specific instance.

Percept Sequence

This is a list of all percepts received by an agent till date.

Agent Function

This is a map from the precept sequence to an action.

3.4.2 Rationality

Rationality is a feature that inculcates a sense of responsibility, sensibility and judgment. It empowers the agent to perform expected actions after perceiving. The agent must perform all the actions to maximize its performance measure with respect to the percept sequence and the knowledge base that it has. Therefore, we can say that rationality of an agent depends on the following:

1. **Agent's performance** measure that gives the degree of success.
2. **Agent's percept sequence** received so far.
3. Agent's p**rior knowledge** about the environment.
4. Agent's a**ctions** that can be performed.

A rational agent (refer **Fig. 3.3**) always performs the right action in the given percept sequence to maximize performance. Thus, a problem solved by an agent is characterized by performance measure, environment, actuators, and sensors (PEAS).

FIGURE 3.3 Rational Agent

3.4.3 Structure of Intelligent Agents

The structure of an intelligent agent can be given as,

Agent = Architecture + Agent Program

where architecture is the machinery on which an agent works, and agent program is an implementation of an agent function.

3.4.4 Types of Agents

In this section, we will discuss different types of agents.

Simple Reflex Agents

Simple reflex agents choose actions based only on the current percept. They are rational only if they make a correct decision based on the current precept. Their environment is completely observable.

They work using **condition-action rule** that maps a state (condition) to an action. If the condition is true, then the action is taken, else not. However, the problem with a simple reflex agent is that they succeed only when the environment is fully observable. If the environment is partially observable, then the agent may get stuck in infinite loops. In such a situation, the agent can escape the loop only if it can randomize its actions. Other issues with such agents are given below.

1. Possess very limited intelligence
2. Has knowledge of any state other than the current state.
3. Any change in the environment will eventually end up updating the collection of rules.

Model-Based Reflex Agents

Model reflex agents (refer **Fig. 3.4**) use a model of the world to choose their actions. For this, they need to maintain an internal state. Here, **model** means knowledge about ‘how things happen in the world’.

FIGURE 3.4 Model reflex agent

Internal state represents unobserved aspects of the current state depending on percept history.

To update the state, information is required to understand,

- How the world evolves and
- How the agent's actions affect the world.

Goal-Based Agents

Goal means a description of desirable situations. Goal-based agents (refer **Fig. 3.5**) choose their actions to achieve goals. This provides more flexibility than reflex agent as the knowledge supporting a decision is explicitly modeled and permits any sort of modifications.

FIGURE 3.5 Goal-based agent

Utility-Based Agents

Many a time, goals are either conflicting or are difficult to achieve. In such situations, utility-based agents (refer **Fig. 3.6**) can be used to achieve goals that are more important than others. These agents choose actions based on a preference (utility) for each state.

FIGURE 3.6 Utility-based agents

However, at times, achieving the desired goal is not enough and happiness of the agent is also important. In such a scenario, utility describes how **'happy'** the agent is. A utility agent chooses the action that maximizes the expected utility, that is, the associated degree of happiness.

Learning Agent

A learning agent (refer **Fig. 3.7**) learns from its past experiences. It is designed to possess learning capabilities. Initially, it starts working with basic knowledge and then acts and adapts automatically through learning.

FIGURE 3.7 Learning agent

Such an agent has four main conceptual components, as follows:

Learning element, which is responsible for making improvements by learning from the environment

Critic (refer **Fig. 3.8**) is **t**he element that provides feedback to the learning element, describing how well the agent is doing with respect to a fixed performance standard.

Performance element chooses a particular external action to be taken.

Problem generator suggests actions that result in new and informative experiences.

FIGURE 3.8 Critic

3.4.5 The Nature of Environments

When building an artificially intelligent system, some programs would be confined to keyboard input, database, computer file systems and character output on a screen while other software agents (like software robots or softbots) would operate in unlimited domains. The simulator for a softbot has a **very detailed, complex environment**. The software agent must take a specific action by choosing from a long array of actions in real time.

For example, a softbot whose job is to scan customer preferences and display them relevant items works in the **real** as well as an **artificial** environment.

The most famous **artificial environment** is the **Turing Test environment**, in which artificial agents are tested with a real agent (on equal ground). The software agent is trained to perform as well as a human.

Turing Test

Turing test is widely used to determine the success of an intelligent behaviour of a system. Two persons and a machine or intelligent system or software agent to be evaluated participate in the test. One of the two persons becomes the tester. The other person and the software agent are then made to sit in different rooms. The tester does not know who is the machine and who is a human. He asks them questions by typing and sending them to both, and receives typed responses.

The aim of this test is to fool the tester. If the tester fails to determine whether the reply is coming from a machine or a human, then the machine is said to be intelligent.

3.4.6 Types of Environments

As stated earlier, environment is the place where the agent would work. It gives rewards and specifies states and actions of the agents. Although environment in an AI system is quite vast, we can identify a small number of dimensions to categorize them as given below.

Discrete/Continuous

If an environment has a limited number of distinct, clearly defined, states, then it is said to be discrete. For example, a software agent designed to play chess game works in a discrete environment as there are limited moves that a player can make. However, a self-driving car operates in a continuous environment.

While a discrete environment has a finite number of percepts and actions that can be performed within it, a continuous environment has no such constraints.

Known Vs Unknown

Known and unknown are based on an agent's state of knowledge to perform an action and they are not actually a feature of an environment. In a known environment, the agent knows the results for all actions but in an unknown environment, the agent has to learn how it should work to

perform an action. Usually, a known environment is partially observable and an unknown environment is fully observable.

While known environments are good examples of **exploitation, unknown environments are best for exploration. Reinforcement learning** is extensively applied in an unknown environment.

Observable/Partially Observable

If an agent can determine the complete state of the environment at each time point from the percepts, then it is said to be observable; otherwise, it is only partially observable. Correspondingly, in an unobservable environment, the agent has no sensors. In a fully observable environment, the complete state of the environment *relevant to the choice of action* of the agent can be captured using its sensors.

While a fully observable environment (refer **Fig. 3.9**) does not require to maintain the internal state to keep track history of the world, a not fully observable environment, on the other hand, must maintain an internal state to keep track of the world.

An environment with sensors can be partially observable because of noise, inaccuracy of the sensors, due to the framework of the task itself or because parts of the state are missing from the sensor data.

For example, the classic chess game is fully observable as one agent can perceive the positions and the moves of the other agent. But in the **Kriegspiel** version of chess, the game environment is partially observable as only invalid moves can be observed.

FIGURE 3.9 Observable/Partially Observable Environment

Static/Dynamic

A static environment does not change when an agent is acting, but a dynamic environment does. Agents in static environments, therefore, do not take into account the state of the environment during the action and are thus easy to deal with. However, in a dynamic environment, agents must consider the world while performing each action. For example, self-driving cars work in a dynamic environment, but games like crossword puzzles and chess operate in a static one.

Between these two extremes, we can also have **semi-dynamic environments** which do not change with the passage of time but the agent's performance score does.

Single Agent/Multiple Agents

The environment may contain other agents which may or may not be of the same kind of the agent. For example, vacuum cleaning environment is a single agent but a chess game is a two-agent environment.

In a multi-agent environment, the performance measure of one agent depends on the action performed by another agent. A multi-agent environment (refer **Fig. 3.10**) may be competitive or cooperative. For example, chess is an example of a **competitive** environment and two taxi-driving agents must avoid collisions and work to maximize the performance of both the agents. Thus, it operates in a **cooperative** environment.

FIGURE 3.10 Multi-agent environment

Accessible/Inaccessible

In an accessible environment, the sensory apparatus of the agent has access to the complete state of the environment. This means that it can obtain complete and accurate information about the state's environment, but this is not so in case of an inaccessible environment.

For example, an empty room whose state can be defined by its temperature is an example of an accessible environment, whereas information about an event on the Earth is an inaccessible environment.

Deterministic/Non-deterministic

In a deterministic environment (refer **Fig. 3.11**), the next state of the environment can be easily determined by the current state as well as the actions of the agent. This is not true for a non-deterministic environment. A n**on-deterministic** environment description maximizes an agent's performance for all possible outcome of its actions. Note that in a deterministic, fully observable environment, the agent does worry about uncertainty.

In a **Stochastic** environment, uncertainty about outcomes is quantified in terms of **probabilities**.

FIGURE 3.11 Deterministic/Non-deterministic Environment

For example, the game of ludo is non-deterministic as the dice will generate a number randomly, creating uncertainty in the environment. On the contrary, chess can be considered a deterministic environment to a certain extent as the state of the game can be determined by estimating the other agent's moves although there can be uncertainty due to the other agent in the game. Most real-life situations are so complex that it is not possible to track of all the unobserved aspects. In such cases, we can consider them as deterministic.

Episodic/Non-episodic

In an episodic environment (refer **Fig. 3.12**), each episode has an agent perceiving and then acting. The quality of its action depends just on the episode itself. Subsequent episodes do not depend on the actions performed in the previous episodes.

Episodic environments are, therefore, simpler as the agent does not need to think ahead. There are however, a series of one-shot actions, and only the current percept is required for the action. On the other hand, in a sequential or non-episodic environment, an agent requires memory to store past actions so that it can determine the next best actions. Therefore, the current decision can affect all future decisions.

FIGURE 3.12 Episodic/Non-episodic Environment

3.5 Search

Algorithms

We have read that artificial Intelligence is the study of building rational agents. These agents make use of some or the other search algorithms in the background to achieve their tasks. For example, some single-player games like tile games, Sudoku, crossword, etc., use search algorithms to deduce a particular position in the game.

A search problem consists of the following:

1. **State space** is the set of all possible states that can be attained by an agent.
2. **Start state** is the state from where the searching is done.
3. **Goal test** is a function that checks if the current state is the goal state or not.
4. **Solution** to a search problem is a sequence of actions (also known as **plan)** that transforms the start state to the goal state. This plan is realized using search algorithms.

3.5.1 Types of Search Algorithms

There are many search algorithms but, in this section, we will discuss six of the most popularly used ones. For better understanding, we will categorize them under two headings—informed and uninformed as shown in **Fig. 3.13**.

FIGURE 3.13 Different Types of Search Algorithms

3.5.2 Properties of Search Algorithms

1. **Completeness:** A search algorithm is said to be complete if it returns at least one solution for a particular input.
2. **Optimality:** A search algorithm is said to give an optimal (best) solution if it has the lowest path cost.
3. **Time and space complexity:** Time complexity is the time taken by an algorithm to complete a given task, and space complexity is the maximum storage space required to perform the search operation. A good search algorithm takes less time and space to do its work.

3.6 Uninformed Search Algorithms

A uninformed search, also known as a blind search algorithm, has no additional information about the goal state other than the one provided in the problem definition. The only information they have is on how to traverse or visit the nodes in the tree.

In such algorithms, the machine blindly follows the technique irrespective of whether it is right or wrong, efficient or inefficient. There can be multiple plans to reach the goal state from the start state. All these paths differ by the order and/or length of actions.

In this section, we will read about depth first search, breadth first search and uniform cost search algorithms. Each of these algorithms has the following information.

Problem graph, the start node S to the goal node G.

Strategy, stating the path followed by the graph reach G.

Fringe, which is a data structure to store all the possible states (nodes) that can be reached from the current state.

Tree, that depicts path while traversing to the goal node.

Solution plan, that specifies the sequence of nodes from S to G.

Path/Step cost are integers that represent the cost to move from one node to another node.

3.6.1 Depth First Search (DFS)

In this search algorithm, the tree is traversed from the root node. The key will be searched till the leaf node of a particular branch. If the key is not found, then the search process backtracks to the point from where the other branch was left unexplored. This process is repeated for that other branch either until the key is found or the entire tree is completely traversed.

FIGURE 3.14 Depth-first search

In **Fig. 3.14**, we can clearly visualize the working of a DFS algorithm. Here, searching starts from the root node A and then traverses nodes B, D and H. Now, H is the leaf node, so we retrace the path to reach node B to traverse its unexplored branch. Nodes E and I are thus traversed. Now, I is the leaf node, so again, the path is retraced to follow the unexplored branch of node E. As a result, node J is visited and then it is found that all nodes in branches of node B have been traced. So, the algorithm will

move further by exploring the other untraced branch of the root node. Here, node C, followed by nodes F, K and G are explored.

From these observations, we can conclude that DFS algorithm occupies a lot of memory space, and takes a lot of time to execute when the solution is at the bottom or end of the tree. DFS is implemented using LIFO, that is, stack data structure.

Time complexity of the algorithm can be given as, $O(b^{m})$. It is e*quivalent to the number of nodes traversed in DFS. Correspondingly,* space complexity is e*quivalent to how large can the fringe get*. It is given as O(bm), where

b is the maximum branching factor in a tree.

d defines the depth of the least-cost solution.

m specifies the maximum depth of the state space. It maybe infinity.

Note that path = the depth of the search tree = the number of levels of the search tree = number of nodes in level .

DFS algorithm explores as far as possible along each branch before backtracking.

Though the DFS algorithm is complete if the search tree is finite and a solution exists, **it** is not optimal as the number of steps in reaching the solution, or the cost spent in reaching it is high.

Advantages

1. It uses less memory as it stores only the nodes on the path from the root node to the current node.
2. It takes less time to reach the goal node/state as compared to the BFS algorithm.

Disadvantages

1. Sometimes, many states keep recurring. In such a scenario, there is no guarantee of finding the solution.
2. In certain cases, the algorithm may get stuck in an infinite loop while going deep searching for nodes in the tree. A possible solution to this problem is to choose an appropriate value of cut-off depth. If the ideal cut-off is *d*, and the chosen cut-off is less than *d*, then algorithm fails. But, if the chosen cut-off is more than *d*, the execution time increases.
3. Complexity of the algorithm depends on the number of paths.
4. DFS cannot check duplicate nodes.

Example: Explore the path that will be explored using the DFS algorithm to reach node G from S.

DFS creates the same set of nodes as breadth-first method, different order.

Solution: To explore the path, a search tree for the graph will be created. Since DFS traverses the tree using the ‘deepest node first’ technique, it would always pick the deeper branch until it reaches the solution or until all nodes of the current branch have been traversed. The traversal is shown in blue arrows.

The DFS path can be given as, S -> A -> B -> C -> G

Example: Consider the graph given below and state its DFS traversal.

Solution: We start with node 0. Exploring its branch to node 1, we move to node 2 and node 4. From node 4, we backtrack to node 3. Hence, the DFS path can be given as, 0 -> 1 -> 2 -> 4 -> 3.

3.6.2 Depth-Limited Search Algorithm (DLS)

DLS is similar to the DFS algorithm with an addition of a predetermined limit. This limit helps to overcome the limitations of the infinite path in the DFS. In DLS, the node at the depth limit will be treated as it is a leaf node (or a node with no successor nodes).

The DLS algorithm is terminated in any of the three cases:

1. First, if the problem does not have any solution. This is known as standard error failure.
2. Second, if the problem does not have any solution within a given depth limit. This is known as cut-off failure.
3. Third, when the solution is found.

For example, in the tree given in **Fig. 3.15**, if level is limited to 2, then the search technique will not go to level 3. Therefore, nodes E, F, G and H will not be traversed.

Thus, the DLS search algorithm is complete if the solution is present within the specified depth limit. While time complexity of DLS algorithm is **$O(b^{\ell})$**, space complexity is O**$(b \times \ell)$,** where l is the depth limit and d is the depth of the tree.

FIGURE 3.15 Depth limited search

Advantages

1. It is a memory efficient algorithm as it consumes less space.
2. The algorithm takes less time to execute.
3. It terminates in finite time.

Disadvantages

1. It is an incomplete algorithm as we may not be able to get the solution every time we do the search (even if the solution exists, due to limit constraint).
2. If multiple solutions of a problem exist, then DLS may not be able to find the optimal solution. In other words, the algorithm is not optimal even if $\ell > d$.

Example: Traverse the given tree to search node H using DLS with predefined limit as 2.

Solution: We start with Node A at level 0.

The search process then continues to explore nodes B, C, D, and E at level 1.

Now, we go to level 2 and explore the child nodes of node B. When H is not found there, a backtrack is done to level 1.

Child nodes of node C are searched to find node H. Finally, node H is found at level 2 as a child node of C and the algorithm terminates.

Example: Traverse the given tree to search node H using DLS with predefined limit as 2.

Solution: In the tree, node H is not present till level 2, so the algorithm terminates returning a cutoff failure. The path traversed for searching is marked with a dashed line.

This image explains the DLS implementation and could be referred to for better understanding.

3.6.3 Breadth First Search (BFS)

In this algorithm, nodes in the tree are traversed breadthwise to reach the goal node. Searching begins from the root node and expands the suc-

cessor node breadthwise before traversing depth-wise.

As evident from **Fig. 3.16**, a BFS algorithm starts from the root node A and then traverses node B followed by node C. This means that children of the root node are traversed before exploring other nodes on any branch. After traversing all nodes at a particular level, control then passes to the next level. Therefore, after node C, all nodes from D to G are traversed. This is followed by traversal of nodes H to K. Thus, the BFS algorithm explores all neighbour nodes at the present depth before moving the nodes at the next depth level.

FIGURE 3.16 Breadth-first search

FIGURE 3.17 BFS Tree starting from a random node

In **Fig. 3.17**, BFS algorithm selects a random initial node (also known as source node or root node) and traverses the graph layer-wise in such a way that all the nodes and their respective children nodes are visited and explored. Here,

Visiting a node means to visit or select a node. And,

Exploring a node means exploring the adjacent nodes (child nodes) of the selected node.

Advantages

1. BFS will find a solution if it exists.
2. If a given problem has multiple solutions, then BFS will give the minimal solution requiring least number of steps.
3. The architecture of the BFS algorithm is simple and robust.
4. It constructs the shortest path of traversing through the nodes of the graph so that the graph can be traversed in the smallest number of iterations.
5. The algorithm does not get stuck in an infinite loop.

Disadvantages

1. Since each level of node is saved for creating the next one, it consumes a lot of memory space. Space requirement to store nodes is exponential.
2. It takes less time if the solution is far away from the root node—at the bottom or at the end.
3. Its complexity depends on the number of nodes.

The BFS algorithm is implemented using a FIFO queue. It is a complete algorithm, that is, it finds a solution if it exits. The algorithm is optimal, if step cost = 1 (that is, if either there is no cost or if all step costs are same). Moreover, it has both time and space complexity given by, $O(b^{d})$. Here, b is the branching factor and d is the depth of the tree.

Note that total number of nodes created in the worst case is $b + b^{2} + b^{3} + \ldots + b^{d}$.

*While time complexity is equivalent to the number of nodes traversed in BFS s***pace complexity, on the other hand, it is e***quivalent to how large can the fringe get.* Due to high precision and robust implementation, BFS is used in multiple real-life solutions like P2P networks, Web Crawlers and Network Broadcasting.

Example: Find the BFS traversal from node S to node G.

Solution: The BFS creates a tree and traverses it using the principle 'shallowest node first'. So, at note S, node D will be traversed followed by node G. Thus, **Path is** S -> D -> G

In a BFS algorithm, nodes in the path = the depth of the shallowest solution = number of nodes in level .

Example: Consider the graph. Using node A as the source node, traverse the graph and trace the working of the algorithm using a queue.

Step 1: Insert root node ‘a’ into the Queue.

Step 2: Remove node ‘a’ from the queue, print it and insert the child nodes of ‘a’ in the queue. Thus, nodes ‘b’ and ‘c’ are inserted.

Step 3: The queue is not empty and has node ‘b’ and ‘c’. Since ‘b’ is the first node in the queue, remove it, print it and insert the child nodes of ‘b’ into the queue. That is, insert node ‘d’ and ‘e’.

Repeat these steps until the queue gets empty. Do not insert those nodes in the queue that are already visited.

Breadth-First Search Algorithm Pseudocode

According to the pseudocode, s is the root node of the graph, G. initially, s is inserted in the queue. Then, all child nodes of ‘s’ are marked. These

child nodes are visited after removing ‘s’ from the queue. At each step of the algorithm, child nodes, w are inserted into the queue to further visit its child nodes. The process is repeated under the queue is empty.

Applications of Breadth-First Search Algorithm

BFS is a simple graph traversal method that is widely used in the following areas.

Crawlers in Search Engines

BFS algorithm is used for indexing web pages. In this application, each web page is considered as a node in a graph. The algorithm starts traversing from the source page and follows all the links associated with that page.

GPS Navigation Systems

GPS systems use BFS algorithm to find neighbouring locations.

Find the Shortest Path for an Unweighted Graph

For an unweighted graph, the shortest path can be easily calculated by choosing a path with the least number of edges. BFS algorithm can do this by traversing a minimum number of nodes starting from the source node. Similarly, either breadth-first search or depth-first traversal methods can be used to find a spanning tree.

Broadcasting

In computer networks, data is broken down into small packets before transmission across the communication media. These packets use an algorithm to compute a path to the destination. For messages that are broadcasted across all the nodes in a network, BFS is a preferred choice.

Peer-to-Peer Networking

BFS is used in peer-to-peer network as a traversal method to find all the neighbouring nodes. For example, BitTorrent uses BFS for peer-to- peer communication.

3.6.4 Uniform Cost Search (UCS)

The UCS algorithm is used to find an optimal solution to the goal state when the step costs are not the same. In such a scenario, the algorithm computes the cumulative cost to expand each node from the root node to the goal node. It neither traverses depth nor breadth, but searches for the next node with the lowest cost. Sorting is done in increasing cost of the path to a node so that the algorithm can explore paths in the increasing order of cost. UCS always expands the node with the least cost. It is identical to BFS if each transition has the same cost. Remember that,

Cost of a node is defined as:

cost(node) = cumulative cost of all nodes from root

cost(root) = 0

In **Fig. 3.18**, if S is the start node and G is the goal state, then node A is explored as it has the lowest cost (the other node G has a higher cost). Now, node A has two child nodes—B and C. C having the lowest step cost is traversed and from there, node D, followed by G is reached. Though UFS performs well for this algorithm, it may not work with all cases.

Thus, we see that uniform-cost search (UCS) expands nodes based on their path costs form the root node. It can be used to traverse any graph/tree where the optimal cost is the traversal criteria. The uniform-cost search algorithm is implemented by the priority queue, giving maximum priority to the lowest cumulative cost. If you observe carefully, you will find that UCS is equivalent to BFS algorithm if the path cost of all edges is the same.

FIGURE 3.18 Uniform cost search

The UCS algorithm is a complete and optimal algorithm with time and space complexity and space complexity $O(b^{(c/\varepsilon)})$, where ε is the lowest cost and c is the optimal cost.

Advantages

1. It finds an optimal solution by considering the least cost at every state.
2. UCS is complete only if states are finite and if there is no loop with zero weight.
3. UCS is optimal only if there is no negative cost.

Disadvantages

1. The algorithm may get stuck in an infinite loop as it considers only cost and not the number of steps taken to reach the goal state.
2. It explores nodes in every ‘direction’.
3. It does not have any information about the location of the goal state.
4. It requires more space for storing information about nodes.
5. The UCS must explore all paths including the long ones.

Example: Consider the tree given below and its UCS traversal to reach node G from S.

Example: Find the path and cost to move from node S to node G in the graph given below.

Solution: For a better clarity, we can draw an equivalent search tree for the graph. Using UCS, the path with the least cumulative cost is chosen.

Path: S -> A -> B -> G

Cost: 5

3.6.5 Iterative Deepening Depth-First Search (IDDFS)

The iterative deepening algorithm combines the features of DFS and BFS algorithms to find out the best depth limit that can be used to implement the DLS algorithm. IDDFS does so by gradually increasing the limit until the goal is found.

The algorithm performs depth-first search to level 1, starts over, executes a complete depth-first search to level 2 and continues until the solution is found. It never creates a node until all lower nodes are generated. It only saves a stack of nodes.

IDDFS gets benefitted by fast search technique of BFS algorithm and memory efficiency of the depth-first search algorithm. This algorithm is

widely used when search space is large, and depth of the goal node is not known.

Advantages

1. It combines the benefits of BFS and DFS search algorithms—fast search and memory efficiency.
2. The algorithm is complete is if the branching factor is finite.
3. IDDFS is an optimal algorithm if path cost is a non-decreasing function of the depth of the node.

Disadvantages

1. It repeats all the work of the previous phase.
2. It takes more time (exponential) to reach the goal node.
3. The algorithm fails when the BFS fails.

Example: Traverse the given tree using the iterative deepening depth-first search algorithm.

Solution: In the first iteration, node A at level 0 is explored.

In the second iteration, nodes B and C are traversed at level 1.

In the third iteration, nodes D, E, F and G are traversed.

In the fourth iteration, node H is reached.

The algorithm ends when it finds a solution at depth *d*. The number of nodes created at depth *d* is b^{d} and at depth *d-1* is b^{d-1}.

Time complexity of IDDFS algorithm is $O(b^d)$ and its space complexity is O(bd), where b is the branching factor and depth of the tree is d.

In IDDFS, we perform DFS up to a certain 'limited depth,' and keep increasing this 'limited depth' after every iteration.

Example: Consider the tree given below and demonstrate the application of IDDFS.

3.6.6 Bidirectional Search

In a bidirectional search, searching is done **from both the directions simultaneously. It searches forward from initial state and backward from goal state till both meet to identify a common state.**

The path from initial state is concatenated with the inverse path from the goal state. Each search is done only up to half of the total path.

The algorithm finds the smallest path from the source node to the goal node. Thus, bidirectional search replaces single search graph with two smaller sub graphs—one starting from initial or start node to the goal node (termed as forward search) and the other starting from goal node towards the source node (known as backward search).

The search process terminates when the two graphs intersect. Bidirectional search can be guided by a heuristic estimate of remaining distance from source to goal and vice versa. Heuristic means finding the shortest path from the current node in the graph to the goal node.

Example: Consider the graph given below and apply bidirectional search on it to reach goal node 14 from source node 0.

Solution: Two searches are executed simultaneously-, one from node 0 and the other from node 14. Both these searches intersect at node 7. At this point, we have found a path from node 0 to 7 and from node 14 to 7. The search process terminates successfully, thereby avoiding unnecessary exploration.

Advantages

1. It is faster than other algorithms
2. **It avoids unnecessary** exploration of nodes
3. Time and space complexity of searching is $O(b^{d/2} + b^{d/2})$, which is far less than $O(b^{d})$. Here, **b** is the branching factor of the tree and d is the

distance of goal node from the source node.
4. The algorithm drastically reduces searching time by supporting simultaneous searches.
5. It takes less memory capacity to store all the searches.
6. Bidirectional search is complete if BFS is used in both searches.
7. The algorithm is optimal if BFS is used for search and paths have uniform cost.

Disadvantages

1. The algorithm can be used only when the goal state is clearly known.
2. It is difficult to implement.
3. The algorithm must be robust enough to correctly deduce the intersection point where it can terminate. Otherwise, the algorithm may get stuck in an infinite loop.
4. It is very difficult to search backwards through all states.

When to Use Bidirectional Approach?

Bi-directional search can be best used in the following situations:

- When both the starting node and the goal node are unique and completely defined.
- When the branching factor is exactly the same in both directions.

Example: Consider the graph given below and demonstrate the application of bidirectional search from Start forward search from node 2 and backward search from node 11.

Do BFS from both directions.

In the forward search process, nodes 1, 6 and 8 are explored. In the backward search process, mode 7 is explored. But no intersection node is encountered.

Next explore node 3 while doing a forward search and node 3 while doing a backward search. Here, we find intersecting node as node 3, so the algorithm terminates and the path is- 2 -> 1 -> 3 -> 7 -> 11.

3.7 Informed Search Algorithms

The uninformed search algorithms that we have learnt so far had no knowledge about search space. But informed search algorithm, as the name suggests, contains some information about how far we are from the goal, cost of the path, how to reach the goal, etc. This knowledge helps agents to explore less to the search space and reach the goal node more efficiently.

The informed search algorithm which is extensively used in a large space is also known as heuristic search as it uses a heuristic function. The heuristic function takes the current state of the agent as its input and estimates how close the agent is from the goal. Though the heuristic method may not always give the best solution, it is, however, guaranteed to find a good solution in reasonable time.

A heuristic function h(n), calculates the cost of an optimal path between the pair of states. The value of the heuristic function is always positive. We can write,

$h(n) <= h^{\star}(n)$

where h(n) is heuristic cost, and $h^{\star}(n)$ is the estimated cost. This means that the heuristic cost should be less than or equal to the estimated cost.

3.7.1 Pure Heuristic Search

To solve large problems with a large number of possible states, problem-specific knowledge needs to be added to increase the efficiency of search algorithms. In such a situation, heuristic search algorithm performs well.

Heuristic search is the simplest form of heuristic search algorithms that expands nodes based on their heuristic value h(n). The algorithm maintains two lists—OPEN and CLOSED. All nodes that have already been expanded are placed in the CLOSED list, while others that have not been expanded are in the OPEN list.

On each iteration, nodes with the lowest heuristic value are expanded. Then, the heuristic function is applied to the child nodes and they are placed in the open list according to their heuristic value. The shorter paths are saved and the longer ones are disposed. The process continues unit a goal state is found.

In this section, we will discuss two important informed search algorithms —b**est first search algorithm (greedy search) and A⋆ search algorithm.**

3.7.2 Best-First Search Algorithm (Greedy Search)

Greedy best-first search algorithm combines depth-first search and breadth-first search algorithms and always selects the path which appears best at that moment. It uses the heuristic function and best-first search at each step to choose the most promising node. The most promising node is the one that is closest to the goal node. The greedy best first algorithm is implemented by the priority queue.

Heuristic evaluation functions calculate the cost of optimal path between two states.

Best first search algorithm steps:

Step 1: Insert the starting node in the OPEN list.

Step 2: Stop and return failure if the OPEN list is empty,

Step 3: From the OPEN list, remove the node having the lowest value of h(n), and insert it in the CLOSED list.

Step 4: Expand the node (removed in Step 3) and generate its successors.

Step 5: Check each successor to find if it is the goal node or not. If any successor node is found to be the goal node, then return success and terminate the search, else go to Step 6.

Step 6: Check if each successor node is in either OPEN or CLOSED list. If the node is not present in any of the lists, then add it to the OPEN list.

Step 7: Go to Step 2.

Advantages

1. It takes advantage of both BFS and DFS algorithms.
2. Best first search algorithm is more efficient than BFS and DFS algorithms.

Disadvantages

1. In some worst-case scenarios, best first search can behave as an unguided depth-first search.
2. Like DFS, best first search algorithm can get stuck in a loop.
3. Best first search algorithm is not an optimal algorithm.

Example: Consider the tree given below and traverse it using greedy best-first search algorithm.

Solution: Expand the node S and put it in the CLOSED list. Generate its successors and place them in the OPEN list.

Open [A, B], Closed [S]

Iteration 1: Since h(n) for node B is less than that of node A, expand B and place it in CLOSED list. Now only node A is in the OPEN list. Open [A], Closed [S, B]

Iteration 2: Generate successors of node B and place them in the OPEN list. Open [E, F, A], Closed [S, B].

Since node F has the lowest heuristic value among all nodes in the OPEN list, we will expand node F. Now, Open [E, A], Closed [S, B, F]

Iteration 3: Generate successors of node F and place them in the OPEN list. So, Open [I, G, E, A], Closed [S, B, F]

Now, generate succors of node F and place them in OPEN list. Since one of the successors of node F is the goal node, the algorithm returns success and terminates. The path is given by the nodes present in the CLOSED list. Open [I, E, A], Closed [S, B, F, G]

Hence, the final solution path will be: **S----> B----->F----> G**

The worst-case time complexity and space complexity of greedy best first search is $O(b^{m})$ where, m is the maximum depth of the search space. Although the algorithm is complete, it can, at times, behave like an incom-

plete algorithm even if the given state space is finite. This makes the Greedy best first search algorithm not an optimal one.

Example: Apply greedy best first search algorithm on the graph given below to reach node I from node S.

Solution: Add node S in the CLOSED list and place its successors in the OPEN list. Open [A, B, C], Closed [S]

Remove A from the OPEN list as it has minimum h(n), place it in CLOSED list and put its successors in the OPEN list.

Open [B, C, E, D], Closed [S, A]

Remove C from the OPEN list as it has minimum h(n), place it in CLOSED list and put its successors in the OPEN list.

Open [B, E, D, H], Closed [S, A, C]

Remove B from the OPEN list as it has minimum h(n), place it in CLOSED list and put its successors in the OPEN list.

Open [E, D, H, F, G], Closed [S, A, C, B]

Remove H from the OPEN list as it has minimum h(n), place it in CLOSED list and put its successors in the OPEN list. Since I is the successor of node H, the algorithm returns success.

Open [E, D, F, G, I, J], Closed [S, A, C, B, H]

3.7.3 A⋆ Search Algorithm

The A⋆ search algorithm, best-known form of best first search, uses heuristic function h(n), and g(n) which specifies the cost to reach the node n from the start state (refer **Fig. 3.19**). It solves the searching problem efficiently by expanding less search tree to find shortest path and give an optimal result faster. That is, it avoids expanding paths that are already expensive, but expands most promising paths first.

FIGURE 3.19 Calculating Cost of A⋆ Algorithm

The A⋆ search algorithm, uses search heuristic as well as the cost to reach the node. Sum of both the costs gives a number known as a **fitness number**.

At each point in the search space, node having the minimum f(n) is expanded. The algorithm terminates when the goal node is found. The steps of this algorithm can therefore be summarized as given below.

Step1: Put the starting node in the OPEN list.

Step 2: If the OPEN list is empty then return failure and stop.

Step 3: From the OPEN list, choose the node having minimum value of f(n). If this node is the goal node, then return success and stop.

Step 4: Expand the node selected in Step 3 and generate its successors. Put the node in the closed list. For each successor, check if it is present in

the OPEN or CLOSED list. If it is not present in either then compute its f(n) and place it into Open list.

Step 5: Else if the successor node is already in OPEN or CLOSED list, then attach it to the back pointer reflecting the lowest g(n') value.

Step 6: Return to **Step 2**.

Points to Remember

1. **Complete:** A⋆ algorithm is complete as long as:
    1. Branching factor is finite.
    2. Cost at every action is fixed.
2. **Optimal:** A⋆ search algorithm is optimal if it follows the two conditions:
    1. **Admissible:** The first condition requires for optimality in that h(n) should be an admissible heuristic for A⋆ tree search. An admissible heuristic is optimistic in nature.
    2. **Consistency:** The second required condition is consistency for only A⋆ graph-search.

    If the heuristic function is admissible, then A⋆ tree search will always find the least cost path.
3. **Time Complexity:** The time complexity of A⋆ search algorithm depends on heuristic function, and the number of nodes expanded is exponential to the depth of solution d. So, the time complexity is O(b^d), where b is the branching factor.
4. **Space Complexity:** The space complexity of A⋆ search algorithm is **O(b^d).**

Advantages

1. It is better than all other search algorithms.
2. A⋆ search algorithm is optimal and complete.
3. The algorithm can be used to solve very complex problems.

4. It is used by many games and web-based maps to find the shortest path efficiently.

Disadvantages

1. It may not always produce the shortest path since it is based on heuristics and approximation.
2. A⋆ search algorithm is more complex than other search algorithms
3. It uses more memory as it keeps all generated nodes in the memory. This makes it difficult to use for large-scale problems.
4. At times, A⋆ algorithm returns the path which occurred first, and ignores searching all other paths.
5. The efficiency of A⋆ algorithm depends on the quality of heuristic.

Example: Consider the graph given below and traverse it using the A⋆ algorithm.

Note than h(n) will be calculated by adding h(n) and the cost of reaching that node.

Solution:

Initially, we start with node S having h(n) = 5. Therefore, {(S, 5)}

Iteration1: From node S, we can reach node A and node G.

h(n) of node A is 3 and the cost to reach node A from node S is 1. Therefore, f(n) of node A = 4. Similarly, node G has h(n) = 0 and the cost to reach node G from node S is 10. Therefore, f(n) of node G = 10.

{(S--> A, 4), (S-->G, 10)}

Iteration2: Since node A has less f(n) as compared to node G, we will expand node A.

h(n) of node B is 4 and the cost to reach node B from node S is 3. Therefore, f(n) of node B = 7. Similarly, node C has h(n) = 2 and the cost to reach node C from node S is 2. Therefore, f(n) of node C = 2.

{(S--> A-->C, 4), (S--> A-->B, 7), (S-->G, 10)}

Iteration3: Since node C has less f(n) as compared to other nodes, we will expand node C.

h(n) of node D is 6 and the cost to reach node D from node S is 5. Therefore, f(n) of node D = 1. Similarly, node G has h(n) = 0 and the cost to reach node G from node S is 6. Therefore, f(n) of node G = 2.

{(S--> A-->C--->G, 6), (S--> A-->C--->D, 11), (S--> A-->B, 7), (S-->G, 10)}

Iteration 4: Since G is the goal node, the algorithm will return success and terminate giving the final path as, **S--->A--->C--->G** with cost 6.

Example: Consider the graph given below and traverse it from node S to G using A* algorithm.

In the graph, the cost to reach node A from node S is 3 + 5 = 8, the cost to reach node B from S is 6 + 4 = 10 and the cost to reach C is 2 + 10 = 12. Undoubtedly, node A is processed further. Now,

{(S--> A, 8), (S--> B, 10), (S-->C, 12)}

In the next iteration, node is found to be reached from A with a total cost of 3 + 10 = 13.

{(S--> A --> G, 13), (S--> B, 10), (S-->C, 12)}

Now, traversing the S -> B path to reach G, we get total cost = 11 (6 + 5). Therefore,

{(S--> A --> G, 13), (S--> B-->G, 11), (S-->C, 12)}

The final path to G from S is thus, S--> B-->G with a cost of 11.

3.7.4 AO⋆ Algorithm

AO⋆ algorithm is based on problem decomposition in which a problem is broken down problem into smaller pieces.

Therefore, AO⋆ algorithm can be applied only on those algorithms that can be divided into a set of sub problems. Each sub problem can be solved separately. The final result is obtained by combining all the solutions. Usually, AND-OR graphs /trees are used for representing the solution. The decomposition of the problem generates AND arcs.

Figure 3.20 shows an AND-OR graph. It shows that if you want to have breakfast then you can eat bread or have fruits and milk together. The arc denotes AND condition.

FIGURE 3.20 AND-OR graph

3.7.4.1 A⋆ Vs AO⋆

Both A⋆ and AO⋆ algorithms are informed search techniques and use heuristic values to solve the problem.

Although both the algorithms guarantee solution, only A⋆ guarantees an optimal solution. By optimal, we mean the shortest path with the lowest cost. The AO⋆ algorithm does not always provide an optimal solution. This is because AO⋆ does not explore all the solution paths once it finds a solution.

Example: Consider the tree given below and apply the AO⋆ algorithm on it.

Remember that the AO* algorithm always moves towards a lower cost value. The cost function, (F(n)= G (n) + H (n)), where, **H(n) is the heuristic or the estimated value of the node and G(n) is the actual cost or edge value.**

Step 1: From node A, we can either choose D or B-C together. The cost of both the paths are calculated as,

F(A to D)= 1+10 = 11 and

F(A to B and C) = 1 + 1 + 6 +12 = 20

We will choose node D as the next node to traverse as it has a lower cost than path B-C.

Step 2: From D, we are left with only one choice H-I. Now, the cost of H-I from node D is calculated as,

F(D to H-I) = 1+1+ 4 +4 =10

Correspondingly, the cost of H-I from node A is 11 as,

F(A to D to F-E) = 1+10 =11

If the goal state is found on the path from A to H-I, then the other path remains unexplored.

Let us explore the other path from node A to B-C. We had calculated,

F(A to B and C) = 1 + 1 + 6 +12 = 20

Step 1: From B, there are two nodes can be reached- E and F.

F(B-E)= 5+1 = 6 and F(B-F) = 7 + 1 = 8

Step 2: Since cost from **F(B-F)** is more than that of the path B-E, the algorithm moves to node J.

F(E-J) = 1 +1 = 2, which is less than heuristic value 5. So, the new heuristic value from E is 2.

Step 3: We need to update the cost from E to B, which is now,

F(E-B)= 1+2 =3

Therefore, the new heuristic value of B is 3.

Step 4: We need to update the value of A as A is associated with both B and C.

But to update the heuristic value of A, we must know the value of node C.

From C, node G can be reached with a cost of 1. So, the heuristic value of C is now 1 + 1 = 2.

Since cost form C to G = F(C-G) = 2 which is less than its heuristic value, the new heuristic value of C is 2.

Step 5: Now the updated from A- BC that is F(A-BC) = 1+1+2+3 = 7 which is less than F(A - D) = 11.

This shows choosing path A-BC is more cost effective and thus better path A-D.

However, this is possible if the algorithm explores this path as well. But the algorithm will not explore this path. We have just shown this path to demonstrate the weakness of the AO⋆ algorithm.

The AO⋆ does not give optimal performance in all the cases.

3.7.4.2 Algorithm

The AO⋆ algorithm maintains two lists like the A⋆ algorithm.

The **OPEN list** contains the nodes that have been traversed have not been marked as solvable or unsolvable.

The **CLOSE** list contains nodes that have already been processed.

Steps in the Algorithm:

Step 1: Put the initial/ starting node in the OPEN list.

Step 2: Find the most promising solution tree say T0.

Step 3: Select a node n that is both a member of T0 as well as present in the OPEN list. Remove n from OPEN and place it in CLOSE list.

Step 4: If n is the leaf node as well as the goal node, then label n and all its ancestors as solved. If the starting node is marked as solved then return success and exit.

Else if n is not a solvable node, then mark n as unsolvable. If starting node is marked as unsolvable, then return failure and exit.

Step 5: If n is not a leaf node, then expand n. Enumerate all its successors, compute their h(n) values and push them into OPEN.

Step 6: Return to Step 2.

Step 7: Exit.

Advantages

1. It gives optimal solution.
2. It can be used for both OR and AND graph when nodes are traversed in a particular order.

Disadvantages

1. It does not find the optimal solution in every case.
2. Its complexity is more than other algorithms.

3.8 Hill Climbing Algorithm in Artificial Intelligence

Hill climbing is a heuristic search technique that works well on a large set of inputs to find a sufficiently good solution to the problem. This solution

may not be the global optimal maximum. The algorithm can be applied to problems that need to maximize or minimize a given real function by choosing values from the given inputs. For example, in the travelling salesman problem, hill climbing algorithm is used to minimize the distance traveled by the salesman. We know that a **heuristic function** uses all the available information to rank all the possible alternatives at any branching step in the search algorithm and select the best route out of possible routes.

3.8.1 Features of Hill Climbing

The hill climbing algorithm is a variant of **generate and test** algorithm which works in the following manner:

Step 1: Generating all possible solutions.

Step 2: Testing them to see if it is the expected solution.

Step 3: If the solution is found, the algorithm terminates else goes to step 1.

Heuristic search algorithm may not find the optimal solution to the problem but gives a good solution in a **reasonable time.**

Thus, the hill climbing algorithm takes feedback from the test procedure and uses it to generate the next move in the search space.

At any point in the state space, the hill climbing algorithm uses the greedy approach to move the search in the direction that optimizes the cost of function so that an optimal solution can be obtained at the end. To take such decisions, the algorithm uses a heuristic value which indicates how close the current state is to the goal state.

It does not support backtracking in the search space, as it does not remember the previous states.

Remember the following:

1. **Hill climbing = generate-and-test + heuristics**
2. **A node of hill climbing algorithm has two components—state and a value.**
3. **Hill climbing algorithm is mostly used when a good heuristic is available.**
4. **The algorithm does not maintain and handle the search tree or graph. It only keeps a single current state.**
5. Hill climbing is the simplest implementation of a **genetic algorithm**. Although this algorithm is less thorough than the traditional ones, it executes faster when compared with other genetic algorithms.

3.8.2 Types of Hill Climbing

In this section, we will discuss different variants of hill climbing algorithm.

3.8.2.1 Simple Hill Climbing

The algorithm examines the neighbouring nodes and selects the first neighbouring node which optimizes the current cost as the next node. The steps performed in this algorithm can be given as follows.

Step 1: Evaluate the initial state. If it is the goal state, then return success and terminate the process. Else, make initial state as the current state.

Step 2: Repeat Step 2 until the solution state is found or no new operators are present which can be applied to the current state.

1. Select a state that has not been set as the current state. Use this state to produce a new state.
2. Evaluate the new state.
    1. If the current state is the goal state, then return success and stop
    2. If the new state is better than the current state, then set it as the current state and processed further.
    3. If the new state is not better than the current state, then continue in the loop until a solution is found.

Advantage

It takes less time.

Disadvantages

1. Solution is less optimal.
2. It does not guarantee any solution

3.8.2.2 Steepest-Ascent Hill Climbing

The algorithm examines all the neighbouring nodes. It then selects the node closest to the solution state as the next node. Steps to perform steepest-ascent hill climbing can be given as follows:

Step 1: Evaluate the initial state. If it is the goal state, then return success and terminate the process. Else, make initial state as the current state.

Step 2: Repeat Step 2 until either the solution state is found or the current state does not change

1. Select a state that has not been set as the current state.
2. Initialize a new 'best state' equal to current state. Use this state to produce a new state.
3. Evaluate the new state.
    1. If the current state is the goal state, then return success and stop.
    2. If the new state is better than the best state, then make it the best state; else continue with the loop to generate and test another new state.
    3. If the new state is not better than the best state, then make best state as the current state. Continue in the loop until a solution is found.

3.8.2.3 Stochastic Hill Climbing

Unlike the previous two variants, the scholastic hill climbing algorithm does not examine all the neighbouring nodes before deciding which node to select next. Rather, it randomly selects a neighbouring node and determines whether to move to that neighbour or to examine another depend-

ing on the amount of improvement in that neighbour. Steps to be performed in scholastic hill climbing are listed below.

Step 1: Evaluate the initial state. If it is the goal state, then return success and terminate the process. Else, make initial state as the current state.

Step 2: Repeat Step 2 until either the solution state is found or the current state does not change.

1. Select a state that has not been set as the current state.
2. Apply successor function to the current state and generate all the neighbour states.
   Randomly select (or use a probabilistic function) a state from all the neighbouring states generated in 2 (b) and are better than the current state.
3. If the state selected in 2 (c) is the goal state, then return success, else make it the current state and continue with the loop.

3.8.3 State Space Diagram for Hill Climbing

The state-space diagram (**Fig. 3.21**) is a graphical representation of the set of states that can be attained in the search algorithm with respect to the value of the objective function (the function that has to be maximized). While the state space is represented on the X-axis, the values of objective function corresponding to a particular state are marked on the Y-axis.

The best solution is the state space where the objective function has a maximum value (global maximum). The goal of the hill climbing algorithm is to find the global maximum and local maximum.

FIGURE 3.21 State Space Diagram for Hill Climbing

3.8.3.1 Understanding Different Regions in the State Space Diagram

Any hill climbing algorithm cannot reach the optimal or the best state (also known as the global maximum) if it enters any of the following regions in the state space diagrams.

Local maximum is a state which is certainly better than its neighbouring state. At this state, the value of the objective function is higher than its neighbours.

However, there does exist a state which is better than it. This better state is known as the global maximum.

Global maximum is the best possible state in the state space diagram and has the highest value for the objective function.

Plateau/Flat local maximum is a flat region in the state space where neighbouring states have the same value.

Ridge is a special type of local maximum that is higher than its neighbours but itself has a slope.

Current state is the region in the state space diagram where the search is being done currently.

Shoulder is identified as a flat plateau that has an uphill edge.

3.8.3.2 Resolving Problems Posed By Different Regions in Hill Climbing

Local Maximum

We know that at the local maximum, all neighbouring states have a value less than that of the current state. Since hill climbing uses a greedy approach, it will not move to the worse state and terminate itself once local maximum is attained. Therefore, the problem here is that the algorithm will terminate without exploring a better solution that may exist.

To overcome this problem, backtracking technique can be used. A list of visited states is maintained so that if the search reaches an undesirable state, then it can backtrack to a previous state and explore a new path.

Plateau

Plateau is a flat land indicating that all neighbours have the same value. In such a scenario, it is difficult to find the best direction.

To overcome plateaus, a big jump is required to come out of this region. For this, a state far away from the current state is selected randomly to increase the probability of landing in a non-plateau region.

Ridge

Ridge is any point in the space that looks like a peak as movement in all possible directions leads to downward value. In such a situation, the algorithm terminates when it reaches a ridge. It is a special form of local maximum.

To overcome issues related to Ridge, use two or more rules before testing to move in several directions at once. For this, bidirectional search can be used or the algorithm can be moved in different directions.

3.8.4 Simulated Annealing

We have seen that a hill-climbing algorithm may get stuck on a local maximum or may not find an optimal solution while doing a random walk.

Simulated annealing is an algorithm which is both efficient and complete.

Usually, a hill climbing algorithm does not support downward moves but simulated annealing allows such moves to escape local maxima and guarantee both efficiency and completeness.

In simulated annealing, the algorithm picks a random move rather than the best move. If the random move improves the state, then it follows the same path. Otherwise, the algorithm moves downhill and chooses another path.

3.8.5 Applications

The hill climbing algorithm can be applied in the following areas:

Marketing

The hill-climbing algorithm is used by marketing people who need to travel to several cities multiple times in a year to develop the best marketing plans. This problem is popularly known as travelling-salesman problem. The algorithm can help them to optimize the distance covered and reduce their travel times by establishing the local minima efficiently.

Robotics

In robotics, hill climbing algorithm is used to enhance the coordination among different systems and components in robots. It is also used in inductive learning methods for coordinating with multiple robots in a team.

Job Scheduling

Job scheduling is a process in which system resources are allocated to different tasks within a computer system. It is achieved by migrating jobs from one node to another (possibly a neighbouring node). In such a scenario, the hill climbing algorithm can be used to establish the right migration route.

3.8.6 Pros and Cons of Hill Climbing Algorithms

Advantages

1. Hill climbing is useful in routing the related problems. For example, in portfolio management, chip designing, job scheduling, vehicle routing, automatic programming, circuit designing, etc.
2. It can be used in optimization problems, especially when there is limited computational power.
3. The algorithm is more efficient than other search algorithms.
4. It can easily work in cases where the goal is to point out the best suitable state.
5. All in all, hill climbing algorithm is popularly used for optimizing the given problems. It provides outstanding solutions to computationally challenging situations.

Disadvantages

However, the disadvantage of this algorithm is that it may terminate at a Local Minima, Ridge or a Plateau.

3.9 Adversarial Search

Adversarial search is a searching algorithm that has more than one entity. Each of these entities has conflicting goals and objectives. These entities participate in a game-like situation. The strategy or game approach of each player depends on the opponent's move. For example, the game situation and intent of the opponent determines whether a player would take a maximizer role or a minimizer role. While Maximizer, maximizes the gain, Minimizer, on the other hand, makes best efforts to limit the damage and minimize the loss.

While normal search has a single entity, adversarial search on the other hand has more than one entity.

Mathematically, adversarial search is based on the concept of **'game theory'** *which states that a game is played between two players. To complete*

the game, one has to win the game and the other loses automatically.'

Adversarial search can be used for all types of games with predictable as well as unpredictable outcomes by following transparent or blind rules.

Each player considers the movements of other players, then dynamically formulates a strategy to reach the end goals. While playing, players may take unforeseen turns to upset the opponent and win the game. So, a quick reaction is required to counter strategize and win the game. To choose the best optimal strategy in a limited time, techniques like pruning and heuristic evaluation are performed.

FIGURE 3.22 Game Theory

In pruning, the unwanted portions of a search tree that makes no difference in its final result are ignored.

Heuristic evaluation function approximates the cost at each level of the search tree, before reaching the goal node.

3.9.1 Game Scenarios

Adversarial search examines the problem which arises when an agent plans ahead of other agents that are planning against it. It is applied in a multi-agent environment, like in games that are modelled as a search problem and use the heuristic evaluation function.

There are different types of game scenarios which are discussed below.

Games with Perfect Information

Such games are open and transparent as every player has complete information. The players are well aware about the status of the game and the move of the opponent. Examples of such games include chess, checkers, ethello and go.

Games with Imperfect Information

In these games, players do not have any information on the status of the game. They take blind moves by making certain assumptions and predicting the outcomes. Examples of the games include, tic-tac-toe, battleship, blind and bridge.

Deterministic Games

These games are played in a transparent manner. All players can see the moves of the opponents and follow pre-defined rules and pattern. There is no randomness associated with a deterministic game as the rules and patterns are followed very strictly. For example, chess, checkers, tic-tac-toe, go are deterministic games.

Non-Deterministic Games

As the name suggests, non-deterministic games are highly unpredictable. Players take random moves and win by sheer luck or chance. For example, games like poker, monopoly, backgammon fall in this category.

Zero-Sum Games

These are purely competitive games in which one player's gain is compensated by another player's loss. The net value of gain and loss is zero, thus, the name.

Here also, each player has different conflicting strategies. So, each player will either try to maximize the gain or minimize the loss, depending on the game situation, strategy likely to be adopted by the opponent and the player's game plan. This scenario is popularly known as **embedded thinking** in problem resolution in AI. A player can easily backtrack and

change his strategy depending on the direction in which the game progresses. Hence, it is also known as backward reasoning in AI. Chess, tic-tac-toe are games that are played in this group.

TABLE 3.2

3.9.2 Key Terminology

In AI, a game can be treated as a search algorithm. In this section, let us get familiar to commonly used terms in the field of game playing.

1. **Start of the game:** This is the initial state of the game and configurations.
2. **Players:** They are the participants in the game space.
3. **Action:** Actions are the moves made by the players in the game space.
4. **Result:** This is the outcome of the moves of the players.
5. **Terminal Test:** It is a check that determines whether the game is over or not.
6. **Utility:** It is the final outcome of the game in terms of loss or win. The utility function is also known as **objective function** or **payoff function**. The winner may get a price of -1, + 1 or 0.
   (-1): If the PLAYER loses.
   (+1): If the PLAYER wins.
   (0): If there is a draw between the PLAYERS

3.9.3 Game Tree

Like any other tree, the game tree has a root node at the top and multiple nodes below the root. Nodes of this tree are the game states and edges are the moves made by players. Each node represents certain moves of the players and thus the status of the game. Each layer in the tree is either a maximizer or a minimizer. As stated earlier, the maximizer maximizes

the gain and the minimizer minimizes the loss. At each level of the tree, the maximizer at a previous layer will be followed by a minimizer layer. Similarly, a minimizer at a previous layer will be followed by a maximizer layer.

Thus, a player takes a maximizer role or minimizer role depending on the game environment and the opponent's strategy. Let us understand how game playing works.

Example: Tic-Tac-Toe game tree

In the game, there are two players MAX and MIN. Both the players take alternate turns and the game starts with MAX. Each player thinks of an optimal strategy to prevent another one from winning. This implies that MIN is acting against Max in the game. Let's enumerate the process stepwise.

Step 1: Each game has an initial state and more than one player will be playing the game.

Step 2: The root node takes the role of maximizer and fills 'X' in any one of the 3 × 3 vacant cells.

Step 3: The minimizer assesses the move made by its opponent (maximizer) and accordingly place '0' in a vacant cell.

Step 4: The maximizer observes the move made by the minimizer in Step 3 and places a 'X' in any of the vacant cell.

Step 5: Repeat Steps 3 and 4 by alternating moves by minimizer and maximizer, respectively, from their standpoint, either till one of the rows or columns gets filled with 'X' or '0'.

Once the terminal condition is reached, the game is declared to be over. At this point, the measure of utility determines the final outcome—maximizer's win or minimizer's win or draw.

FIGURE 3.23 Game Tree for Tic-Tac-Toe

3.9.4 Uses of Adversarial Search

Adversarial search is widely used in different fields to find optimal solutions in a competitive environment. Some examples are as follows.

1. In the legal system, two advocates argue their stand representing their party's cases. In such a scenario, judges can use adversarial search to analyse and deliver judgment.
2. Business organizations use this search technique while sourcing suppliers and buyers for products/services in a competitive manner.
3. Moreover, this technique can be used in any situation where hard negotiations are required. It helps users to maximize the gain by guiding them to negotiate intelligently and prudently.
4. Adversarial search also forms the base for further development of new techniques like Alpha Beta Pruning.

Adversarial search techniques optimize the time, effort and cost required to take complex decisions.

3.9.5 Mini-Max Algorithm

Mini-max algorithm is a recursive or backtracking algorithm which is used in decision-making and game theory. It takes the optimal move to search through the game-tree. Games like chess, checkers, tic-tac-toe, go and other two-player games use the min-max algorithm to compute the best decision for the current state.

The two players named MIN and MAX compete to let the opponent get the minimum benefit and themselves earn the maximum benefit. So, MAX will always choose the maximum value and MIN will select the minimized value.

The algorithm performs a depth-first search to explore the complete game tree. It proceeds all the way down to the terminal node of the tree, then backtracks the tree.

3.9.5.1 Pseudo-Code for MinMax Algorithm

Pseudocode

3.9.5.2 Working of Min-Max Algorithm

In the minimax algorithm, DFS is used to reach the terminal or the leaf nodes where terminal values are specified. Once we reach those nodes, the algorithm backtracks the tree until the initial state occurs. Given below are the steps involved in solving the two-player game tree.

Step 1: The algorithm generates the game tree and applies the utility function to get the utility values for the terminal states.

In the tree given in **Fig. 3.24**, node A is the root node that is also the initial state of the tree. Assuming that maximizer takes the first turn, then set the worst-case initial value as -infinity. For minimizer, take the worst-case initial value equal to +infinity.

FIGURE 3.24 Min-Max Algorithm Step 1

Step 2: Starting from the layer above the terminal nodes, we will find the utilities value for the maximizer.

Note that the initial value of maximizer is -∞, so each value in terminal state is compared with the initial value of Maximizer. The higher value is then set as the new value of the maximizer as shown in the figure.

For node D, 4 is greater than both -1 as well as -∞. Therefore, the utility of D is 4.

For Node E, 6 is greater than both 2 as well as -∞. Therefore, the utility of E is 6.

For Node F, -3 is greater than both -5 as well as -∞. Therefore, the utility of F is -3.

For Node G, 7 is greater than both 0 as well as -∞. Therefore, the utility of F is 7.

FIGURE 3.25 Min-Max Algorithm Step 2

Step 3: In the next step, minimizer finds the utility value of nodes B and C by comparing values of the nodes with +∞.

For Node B, 4 is less than both 6 as well as ∞. Therefore, the utility of B is 4.

FIGURE 3.26 Min-Max Algorithm Step 3

For Node C, -3 is less than both 7 as well as ∞. Therefore, the utility of C is -3.

FIGURE 3.27 Min-Max Algorithm Step 4

Step 4: During maximizer's turn, maximum of all nodes value is selected as the maximum value for the root node. Therefore,

For node A, the utility value is 4 because max(4, -3)= 4

3.9.5.3 Properties of Mini-Max Algorithm

1. **Complete:** The Min-Max algorithm is complete as it always finds a solution if one exists in the finite search tree.
2. **Optimal:** The algorithm gives an optimal solution if both the opponents are playing optimally.
3. **Time complexity:** Since the algorithm uses DFS technique, the time complexity is **O(bm)**, where b is branching factor (number of child nodes of a node) of the game-tree, and m is the maximum depth of the tree.
4. **Space complexity:** Space complexity of Mini-max algorithm is same as that of DFS which is **O(bm)**.

Advantages

1. The search space is thoroughly assessed to find an optimal solution.
2. It supports decision-making.
3. New and smart machines are developed with this algorithm.

Disadvantages

1. The minimax algorithm gets very slow for complex games such as chess, go, etc. as these games have a high branching factor, and there are lots of choices at each node.
2. If there are time and space constraints, then applying this algorithm would be difficult as more time and space is required to explore the entire tree.

However, this limitation can be improved by using **alpha-beta pruning.**

3.10 Alpha-Beta Pruning

Alpha-beta pruning is the modified as well as optimized version of the minimax algorithm. We have seen that the minimax search algorithm has a large number of game states to examine. The alpha-beta pruning technique cuts the number of nodes to be explored to half. This is done by using the pruning process in which the correct minimax decision is taken without checking each node of the game tree. In the alpha beta pruning algorithm, we take two threshold parameters—alpha and beta.

Alpha is the best or the highest-value so far at any point along the path of maximizer. Its initial value is **-∞**. **Beta; on the other hand, is the** lowest-value found so far at any point along the path of minimizer. Its initial value of beta is +∞.

Alpha-beta pruning can be applied at any depth of a tree. The algorithm removes all the nodes which do not affect the final decision. Therefore, removing such nodes speeds up processing of the algorithm. Before executing the alpha-beta pruning algorithm, remember the following points.

1. The main condition that must be met for alpha-beta pruning is: α >= β.
2. The Max player will update the value of alpha.
3. The Min player will update the value of beta.
4. While backtracking the tree, the value of the node (and not α, β) is passed to upper nodes.
5. The values of α and β are passed to the child nodes.

3.10.1 Pseudo-Code for Alpha-Beta Pruning

Example: Consider the game play tree given in **Fig. 3.28** and apply alpha-beta pruning algorithm on it.

Step 1: For node A, set α= -∞ and β= +∞. Pass down these values to node B and node D. Now, all three nodes have α= -∞ and β= +∞.

FIGURE 3.28 Alpha-beta pruning algorithm step 1

Step 2: Node D will play the role of maximizer and will therefore choose a value of α. The value of α at node D = 3 since max (2, 3) = 3.

Step 3: Algorithm backtracks to node B which plays the role of a minimizer. So, node B will set the value of β. Recall that β= +∞ at B. After comparing this value with the available subsequent node's value, we set β = 3 as min (∞, 3) = 3. The algorithm also passes values α= -∞, and β= 3 to its successor node E.

FIGURE 3.29 Alpha-beta pruning algorithm after step 3

Step 4: At node E, maximizer will update the value of alpha. The current value of alpha will be compared with 5. Since max (-∞, 5) = 5, α = 5 and β= 3, where α>=β, so the right successor of E will be pruned, and algorithm will not traverse it, and the value at node E will be 5.

FIGURE 3.30 Alpha-beta pruning algorithm step 4

Step 5: The algorithm backtracks the tree to node A from node B. At this node, the value of alpha is updated to 3 as max (-∞, 3) = 3. Now α and β (+∞) values are passed to the right successor of A which is Node C. At node C, α=3 and β= +∞, and these values are passed to node F.

Step 6: At node F, the value of α is compared with the value of its left child which is 0. Since, max(3,0)= 3, and then with right child which is 1, and max(3,1)= 3. The node value of F becomes 1.

FIGURE 3.31 Alpha-beta pruning algorithm after step 5

Step 7: The value of node F is returned to node C. At C, α= 3 and β= +∞. Since node B plays the role of minimizer, value of beta is set to 1 as, min

(∞, 1) = 1. Now, the condition $\alpha >= \beta$ is met. Therefore, the right child of C which is G is pruned. Thus, the algorithm will not compute the entire sub-tree G.

FIGURE 3.32 Alpha-beta pruning algorithm after step 7

Step 8: Node C returns the value of 1 to A. Since, A plays a maximizer, it selects value 3 as max (3, 1) = 3.

FIGURE 3.33 The Final Tree

3.10.2 Worst-Case and Best-Case in Alpha-Beta Pruning

The efficiency of alpha-beta pruning algorithm depends on the order in which each node in the game tree is examined. This order can be of two types.

Worst Ordering

This occurs when the algorithm does not prune any of the leaf nodes. In this case, alpha beta pruning works like the minimax algorithm and consumes more time. The best move occurs on the right side of the tree and the time complexity for such an order is $O(b^{m})$.

Best Ordering

The best or the ideal ordering occurs when many branches in the tree are pruned. The best moves occur at the left side of the tree. Time complexity in ideal ordering is $O(b^{m/2})$.

Thus, we see that the alpha-beta pruning is not a new algorithm, it is just an optimization of minimax algorithm. It reduces the computation time while going deep in the tree at a faster speed by cutting off branches in the game tree which need not be searched as a better move has already been found.

Example: Consider the tree given in **Fig. 3.34** and apply Alpha Beta pruning algorithms on it.

FIGURE 3.34 Tree for Alpha Beta Pruning

Step 1: Start from node A. Set α= -∞ and β= +∞. Pass down these values to subsequent nodes in the tree.

Recall that node **A** will must choose max of **B** and **C**, so we need to first find the value of B. **B** is a minimizer, so have to choose min of **D** and **E**. Hence, we need to first know the value of D.

Step 2: D is a maximizer node, so its value is 5 as, max(-∞,3) = 3 and max(3,5) = 5.

Step 3: The algorithm returns the value of D to B as now $\alpha >= \beta$. The value of B is then set to 5 because min(β,5) = 5 and min(5,6) = 5.

Here, right child of node E node is not evaluated. This is because node B is a minimizer and it already knows a value that is smaller than node E, so it will not explore all the branches of E.

FIGURE 3.35 Alpha Beta Tree after Step 2

FIGURE 3.36 Alpha Beta Tree after Step 3

FIGURE 3.37 The final Alpha Beta Tree after Left Sub Tree is Traversed

Step 4: The value of B is returned to node A. At **A**, α= max(-∞,5) which is 5. The algorithm will then find the value of node C before finalizing the α and β values for node A.

The values α, β are passed down to C, F and G. At **C**, α= 5 and β= +∞. To decide value of C, we need to know the value of node F.

Step 5: At **F**, α= 5 and β= +∞. **F** being a maximizer, sets its α = 5 as max(1,5) = 5 and max(2,5) = 5. And the best value of F is 2. The value of F is returned to node C.

Step 6: At **C**, β = min(+∞,2) = 2. Now, condition α >= β is met and the algorithm breaks without compute the sub-tree of **G**. node C is a minimizer node and has got a value that is less than what it already has. **C** returns a value of 2 to **A**.

Step 7: A being a maximizer node sets its value as 5 because max(5, 2) which is a 5.

FIGURE 3.38 Alpha Beta Tree after Step 4

FIGURE 3.39 Alpha Beta Tree after Step 5

FIGURE 3.40 Alpha Beta Tree after Step 6

3.11 Local Search Algorithms

Local search algorithms start from a prospective solution and then move to a neighbouring solution. They can return a valid solution even if it is interrupted at any time before they end.

3.11.1 Hill-Climbing Search

It is an iterative algorithm that starts with an arbitrary solution to a problem and attempts to find a better solution by changing a single element of the solution incrementally. If the change produces a better solution, an incremental change is taken as a new solution. This process is repeated until there are no further improvements.

function Hill-Climbing (problem), returns a state that is a local maximum.

Disadvantage

This algorithm is neither complete, nor optimal.

3.11.2 Local Beam Search

In this algorithm, there are k number of states at any given time. At the start, these states are generated randomly. The successors of these k states are computed with the help of objective function. If any of these successors is the maximum value of the objective function, then the algorithm stops.

Otherwise, the (initial k states and k number of successors of the states = 2k) states are placed in a pool. The pool is then sorted numerically. The highest k states are selected as new initial states. This process continues until a maximum value is reached.

function BeamSearch(problem, k), returns a solution state.

3.11.3 Simulated Annealing

Annealing is the process of heating and cooling a metal to change its internal structure for modifying its physical properties. When the metal cools, its new structure is seized, and the metal retains its newly obtained properties. In simulated annealing process, the temperature is kept variable.

We initially set the temperature high and then allow it to ‘cool’ slowly as the algorithm proceeds. When the temperature is high, the algorithm is allowed to accept worse solutions with high frequency.

Start

- Initialize k = 0; L = integer number of variables;
- From i → j, search the performance difference Δ.
- If Δ <= 0 then accept else if exp(-Δ/T(k)) > random(0,1) then accept;
- Repeat steps 1 and 2 for L(k) steps.
- k = k + 1;

Repeat steps 1 through 4 till the criteria is met.

End

3.11.4 Travelling Salesman Problem

In this algorithm, the objective is to find a low-cost tour that starts from a city, visits all cities en-route exactly once and ends at the same starting city.

Start

Find out all (n -1)! Possible solutions, where n is the total number of cities.

Determine the minimum cost by finding out the cost of each of these (n -1)! solutions.

Finally, keep the one with the minimum cost.

end

FIGURE 3.41 Solving Travelling Salesman Problem

3.12 Single Agent Path-Finding Problems

Games such as 3X3 eight-tile, 4X4 fifteen-tile and 5X5 twenty-four tile puzzles are single-agent-path-finding challenges. They consist of a matrix of tiles with a blank tile. The player is required to arrange the tiles by sliding a tile either vertically or horizontally into a blank space with the aim of accomplishing some objective.

Other examples of single agent pathfinding problems are Travelling Salesman Problem, Rubik's Cube and Theorem Proving.

3.12.1 Search Terminology

1. **Problem space:** It is the environment in which the search takes place. (A set of states and set of operators to change those states)
2. **Problem instance:** It is Initial state + Goal state.
3. **Problem space graph:** It represents problem state. States are shown by nodes and operators are shown by edges.
4. **Depth of a problem:** Length of a shortest path or shortest sequence of operators from Initial State to goal state.
5. **Space complexity:** The maximum number of nodes that are stored in memory.
6. **Time complexity:** The maximum number of nodes that are created.
7. **Admissibility:** A property of an algorithm to always find an optimal solution.
8. **Branching factor:** The average number of child nodes in the problem space graph.

9. **Depth:** Length of the shortest path from initial state to goal state.

3.12.2 Brute-Force Search Strategies

They are most simple, as they do not need any domain-specific knowledge. They work fine with small number of possible states.

Requirements:

- State description
- A set of valid operators
- Initial state
- Goal state description

Comparison of Various Algorithms Complexities

Let us see the performance of algorithms based on various criteria (**Table 3.3**).

TABLE 3.3



PROGRAMMER'S ZONE

Write a Python code to implement Depth First Search Technique

OUTPUT

Write a Python code to implement Breadth First Search Technique

OUTPUT

Write a program to implement Best First Search Technique

OUTPUT

0 1 3 2 8 9

Write a program to implement minmax algorithm

OUTPUT

The optimal value is : 12

Write a Python code to implement Local Beam Search algorithm

OUTPUT



KEY TERMS

Reasoning: It is the set of processes that is used making decisions, and prediction.

Learning: It is the process of gaining knowledge or skill by studying, practising, being taught or experiencing something.

Problem solving: It is the process in which one tries to find the desired solution in the present situation by following the path which is blocked by known or unknown hurdles. Problem solving uses decision-making to select the best suitable alternative to reach the desired goal.

Perception: It is the process of acquiring, interpreting, selecting and organizing sensory information.

Rationality: It is a feature that inculcates a sense of responsibility, sensibility and judgment.

State space: It is the set of all possible states that can be attained by an agent.

Start state: It is the state from where the searching is done.

Goal test: It is a function that checks if the current state is the goal state or not.

Solution to a search problem: It is a sequence of actions (also known as plan) that transforms the start state to the goal state. This plan is realized using search algorithms.

Fringe: It is a data structure to store all the possible states (nodes) that can be reached from the current state.

State-space diagram: It is a graphical representation of the set of states that can be attained in the search algorithm with respect to the value of the objective function.

Plateau/Flat local maximum: It is a flat region in the state space where neighbouring states have the same value.



RECAPITULATION POINTS

1. Simple reflex agents choose actions based only on the current percept.
2. A learning agent learns from its past experiences. It is designed to possess learning capabilities. Initially, it starts working with basic knowledge and then acts and adapts automatically through learning.
3. Turing test is widely used to determine the success of an intelligent behaviour of a system.
4. Environment is the place where the agent would work. It gives rewards and specifies states and actions of the agents.
5. A uninformed or blind search algorithm has no additional information about the goal state other than the one provided in the problem definition. The only information they have is on how to traverse or visit the nodes in the tree.
6. Depth-limited-search (DFS) creates the same set of nodes as Breadth-First method, only in the different order.
7. In Depth-First-search (DLS), the node at the depth limit will be treated as it is a leaf node (or a node with no successor nodes).
8. In BFS algorithm, nodes in the tree are traversed breadthwise to reach the goal node. Searching begins from the root node and expands the successor node breadthwise before traversing depth-wise.
9. The uniform cost search (UCS) algorithm is used to find an optimal solution to the goal state when the step costs are not the same. In such a scenario, the algorithm computes the cumulative cost to expand each node from the root node to the goal node. It neither traverses depth nor breadth, but searches for the next node with the lowest cost.
10. The iterative deepening algorithm combines the features of DFS and BFS algorithms to find out the best depth limit that can be used to implement the DLS algorithm. IDDFS does it by gradually increasing the limit until the goal is found.
11. In bi-directional search, searching is done from both the directions simultaneously. It searches forward from initial state and backward from goal state till both meet to identify a common state.
12. The heuristic function takes the current state of the agent as its input and estimates how close the agent is from the goal. Though the heuris-

tic method may not always give the best solution, it is, however, guaranteed to find a good solution in reasonable time.

13. A* algorithm solves the searching problem efficiently by expanding less search tree to find shortest path and give an optimal result faster. It avoids expanding paths that are already expensive, but expands most promising paths first.
14. AO* Algorithm is based on problem decomposition in which a problem is broken down problem into smaller pieces.
15. Hill climbing is a heuristic search technique that works well on a large set of inputs to find a sufficiently good solution to the problem.
16. In simulated annealing, the algorithm picks a random move rather than the best move. If the random move improves the state, then it follows the same path. Otherwise, the algorithm moves downhill and chooses another path.
17. Mini-max algorithm is a recursive or backtracking algorithm which is used in decision-making and game theory.
18. Local search algorithms start from a prospective solution and then move to a neighbouring solution. They can return a valid solution even if it is interrupted at any time before they end.



ANSWER THE QUESTIONS

1. When is a machine said to be artificially intelligent?
2. Describe the components of intelligence.
3. Define learning and give different types of learning.
4. What are the different types of agents?
5. What is Turing test?
6. Explain the term ‘environment’ in the context of artificial intelligence.
7. Give the properties of a search algorithm.
8. When can DFS algorithm get stuck in an infinite loop? How can it be overcome?
9. Under what circumstances does the DLS algorithm terminate?
10. How does UCS work? When is it the same as BFS?
11. List the merits and demerits of UCS algorithm.

12. Give the pros and cons of A* algorithm.
13. Explain the different regions in the State Space Diagram.
14. Discuss the different types of game scenarios.
15. Explain the mini max algorithm.
16. Write short notes on:
    1. Bi-directional search
    2. Heuristic search
    3. Hill climbing algorithm
    4. Local search algorithms
17. Differentiate between the following:
    1. Inductive and deductive reasoning.
    2. Human and machine intelligence
    3. Competitive and cooperative multi-agent environment
    4. Episodic and non-episodic environment
18. Apply the specified algorithms on the graphs given below.
    1. Breadth First Search (BFS)

    2. Uniform Cost Search (UCS)

3. Uniform Cost Search (UCS)

4.

5. Best First Search

Heuristic Values: A=2; B=3; C=1; D=4; S=10; G=0

6.

7. Bi-directional search

8. Breadth First Search

19. Consider the graph given below and find the cost of reaching G from S using Uniform Cost Search.

20. Compute the route to Romania, using the straight-line distance as the heuristic value. If the goal is Bucharest, and initial position is Arad, then compute the straight-line distances to Bucharest, in the graph giv-

en below.

21. Consider the graph given below. The numbers written on the edges represent the distance between the nodes and the numbers written on nodes represent the heuristic value. Using A* Algorithm, find the most cost-effective path to reach from start state A to final state J.

22. Consider the graph given below. Find a path from A to D using A* algorithm.

23. Give the traversal of the graph given below using DLS algorithm with limit = 1

24. Identify the terms.

25. Consider the game tree given below and compute the utility value of root node using the minimax algorithm.

26. Consider the graph given below and apply AO* algorithm on it.



FILL UP

1. _____ intelligence is mostly used by players and dancers.
2. Two types of reasoning are _____ and _____.
3. If something is true for a particular class of things in general, it is also true for all members of that class. This happens in case of _____ reasoning.
4. _____ is a process of gaining knowledge or skill by studying, practicing, being taught or experiencing something.
5. _____ learning promotes learning by remembering sequences of events that one has witnessed or experienced.
6. A _____ agent uses bit strings as its programs and actions.
7. _____ is a feature that inculcates a sense of responsibility, sensibility and judgment.
8. _____ means a description of desirable situations.
9. _____ is widely used to determine the success of an intelligent behaviour of a system.
10. _____ is the place where the agent would work.
11. In a _____ environment, the next state of the environment can be easily determined by the current state as well as the actions of the agent.
12. In a stochastic environment, uncertainty about outcomes is quantified in terms of _____.
13. Complexity of DFS algorithm depends on the _____.
14. UCS is equivalent to _____ algorithm if the path cost of all edges is the same.
15. The _____ algorithm combines the features of DFS and BFS algorithms.
16. _____ means finding the shortest path from the current node in the graph to the goal node.
17. A _____ function uses all the available information to rank all the possible alternatives at any branching step in the search algorithm and select the best route out of possible routes.
18. The _____ diagram is a graphical representation of the set of states that can be attained in the search algorithm with respect to the value

of the objective function.

19. _____ is identified as a flat plateau that has an uphill edge.
20. In_____, the unwanted portions of a search tree that makes no difference in its final result are ignored.
21. Players take random moves and win by sheer luck or chance when playing _____ games.
22. _____ is the modified as well as optimized version of the minimax algorithm.
23. In _____ algorithm, there are k number of states at any given time. At the start, these states are generated randomly.
24. _____ is the average number of child nodes in the problem space graph.



STATE TRUE OR FALSE

1. *Interpersonal intelligence* is the ability to distinguish between one's feelings, intentions and motivations.
2. Deductive reasoning uses specific observations to make broad general statements.
3. Motor learning helps to learn by precise movement of muscles.
4. Problem solving is the process of acquiring, interpreting, selecting and organizing sensory information.
5. Performance element provides feedback to the learning element.
6. A known environment is partially observable and an unknown environment is fully observable.
7. A fully observable environment must maintain the internal state to keep track history of the world.
8. Deterministic environment descriptions maximize an agent's performance for all possible outcome of its actions.
9. Game of ludo is non-deterministic.
10. In a non-episodic environment, the current decision can affect all future decisions.
11. DFS cannot check duplicate nodes.
12. Visiting a node means exploring the adjacent nodes of the selected node.

13. Bi-directional search takes less memory capacity to store all the searches.
14. Heuristic function always finds a good solution in reasonable time.
15. All nodes that have already been expanded are placed in the OPEN list.
16. On each iteration, nodes with the highest heuristic value are expanded.
17. An admissible heuristic is optimistic in nature.
18. The AO* does not give optimal performance in all the cases.
19. Hill climbing does not support backtracking in the search space.
20. The value of the objective function should be minimized.
21. Plateau is a special type of local maximum that is higher than its neighbours but itself has a slope.
22. Tic-Tac-Toe is an example of a game with perfect information.
23. Zero-sum games are purely competitive games in which one player's gain is compensated by another player's loss.
24. The alpha-beta pruning technique cuts the number of nodes to be explored to one-fourth.
25. The max player will update the value of beta.
26. The values of α and β are passed to the upper nodes in alpha-beta pruning algorithm.
27. In alpha-beta pruning algorithm, best ordering occurs when many branches in the tree are pruned.
28. The depth of a problem is the length of the longest path from Initial State to goal state.



MULTIPLE CHOICE QUESTIONS

1. _____ *intelligence* is the ability to perceive visual or spatial information, change it and re-create visual images
    1. Bodily-kinesthetic
    2. Spatial
    3. Logical-mathematical
    4. Musical

2. _____ is the set of processes used in making decisions and prediction.
    1. Reasoning
    2. Learning
    3. Perception
    4. Problem solving
3. _____ learning triggers learning by watching and imitating others.
    1. Perceptual
    2. Relational
    3. Observational
    4. Stimulus-response
4. Identify the incorrect statement.
    1. Agent can be anything that is capable of perceiving its environment.
    2. Agents acts upon their environment using sensors.
    3. An agent must perform all the actions to maximize its performance measure with respect to the percept sequence and the knowledge base that it has.
    4. Rationality empowers the agent to perform expected actions after perceiving.
5. _____ agents work using Condition-Action Rule that maps a state to an action.
    1. Simple reflex
    2. Model-based reflex
    3. Goal-based
    4. Utility-based
6. _____ element of the learning agent provides feedback.
    1. Performance
    2. Learning
    3. Critic
    4. Problem generator
7. A _____ environment has a finite number of percepts.
    1. Continuous
    2. known
    3. observable
    4. discrete
8. _____ learning is extensively applied in an unknown environment.

1. Reinforcement
2. Machine
3. Deep
4. Transfer

9. An environment with sensors can be partially observable because of _____.
    1. noise
    2. inaccuracy of the sensors
    3. states are missing from the sensor data
    4. All of these
10. Crossword puzzle is an example of _____ environment.
    1. Static
    2. dynamic
    3. Semi-dynamic
    4. None of these
11. A/An _____ environment has a series of one-shot actions, and only the current percept is required for the action.
    1. episodic
    2. non-episodic
    3. deterministic
    4. non-deterministic
12. _____ is a data structure to store all the possible states (nodes) that can be reached from the current state.
    1. tree
    2. strategy
    3. fringe
    4. path
13. Identify the incorrect statement about DFS.
    1. It occupies a lot of memory space.
    2. DFS is implemented using queue data structure.
    3. Time Complexity of DFS is $O(b^{m})$.
    4. Space complexity of DFS is O(bm).
14. In DLS, the node at the depth limit will be treated as a _____ node.
    1. parent
    2. root
    3. child
    4. leaf

15. _____ search algorithm is used in multiple real-life solutions like P2P networks, Web Crawlers and network broadcasting.
    1. Breadth first
    2. Depth first
    3. Depth limited
    4. Uniform cost
16. The uniform-cost search algorithm is implemented using a _____.
    1. Stack
    2. Queue
    3. Priority queue
    4. Linear array
17. Identify the incorrect statement about UCS.
    1. UCS is complete only if there is no loop with zero weight.
    2. UCS is optimal only if there is no negative cost.
    3. UCS does not have any information about the location of the goal state.
    4. UCS requires minimal space for storing information about nodes.
18. _____ algorithm is used when search space is large, and depth of the goal node is not known.
    1. BFS
    2. DFS
    3. UFS
    4. IDDFS
19. The value of the heuristic function is always _____.
    1. zero
    2. negative
    3. positive
    4. one
20. The A* search algorithm, is the best-known form of _____ search algorithm.
    1. Breadth first
    2. Best first
    3. Depth first
    4. IDDFS
21. The space complexity of A* search algorithm is _____.
    1. O(b^d)

2. O(bd)
3. O(d^b)
4. O(b+d)

22. _____ algorithm uses AND-OR graphs /trees used for representing the solution.
    1. A*
    2. Best First Search
    3. AO*
    4. IDDFS
23. _____ is a flat region in the state space where neighbouring states have the same value.
    1. Plateau
    2. Ridge
    3. Shoulder
    4. Current state
24. _____ search can be used to overcome ridge problems.
    1. Best first
    2. Bidirectional
    3. Depth first
    4. Breadth first
25. _____ search examines the problem which arises when an agent plans ahead of other agents that are planning against it.
    1. Simulated annealing
    2. Adversarial
    3. Hill climbing
    4. Bi-directional
26. Chess is an example of _____.
    1. Games with perfect information and deterministic games
    2. Games with imperfect information and deterministic games
    3. Games with perfect information and non-deterministic games
    4. Games with imperfect information and non-deterministic games
27. The mini-max algorithm performs a _____ search to explore the complete game tree.
    1. Depth first
    2. Breadth first
    3. Best first

4. Adversarial

28. Values of Alpha and Beta are _____.
    1. -∞, +∞
    2. -∞, -∞
    3. +∞, -∞
    4. Not defined



ANSWERS

Fill Up

1. Bodily-kinesthetic
2. inductive and deductive.
3. deductive
4. Learning
5. Episodic
6. Software
7. Rationality
8. Goal
9. Turing test
10. Environment
11. Deterministic
12. Probabilities
13. number of paths
14. BFS
15. iterative deepening
16. Heuristic
17. heuristic
18. state-space
19. Shoulder
20. pruning
21. Non-deterministic.
22. Alpha-beta pruning
23. Local beam search
24. Branching factor

State True or False

1. False
2. False
3. True
4. False
5. False
6. True
7. False
8. False
9. True
10. True
11. True
12. False
13. True
14. True
15. False
16. False
17. True
18. True
19. True
20. False
21. False
22. False
23. True
24. False
25. False
26. False
27. True
28. False

Multiple Choice Questions

1. b
2. a
3. c
4. b
5. a

6. c
7. d
8. a
9. d
10. b
11. a
12. c
13. b
14. d
15. a
16. c
17. d
18. d
19. c
20. b
21. a
22. c
23. a
24. b
25. b
26. a
27. a
28. a